

---

Paper on

# MATRIX FACTORIZATION TECHNIQUES FOR RECOMMENDER SYSTEMS

Yehuda Koren, Robert Bell and Cris Volinsky  
IEEE Computer Society 8(2009): 42-49.

(Citation 5580회, 구글 스칼라 기준)

발표자 : 권성은

# Contents

---

1. Recommender System Strategies
2. Matrix Factorization Methods
3. A Basic Matrix Factorization Model
4. Learning Algorithms
5. Adding Biases
6. Additional Input Sources
7. Temporal Dynamics
8. Inputs with Varying Confidence Levels
9. Netflix Prize Competition

# 1. Recommender System Strategies

---

## 기존 방식 리뷰1

Content filtering approach

Collaborative filtering approach

Neighborhood methods

Latent factor models

# 1. Recommender System Strategies

## 기존 방식 리뷰2\_2가지 Collaborative filtering

### Neighborhood methods

- Item간 관계 또는 사용자들간 관계를 계산하는 방식
- Item oriented approach : “neighboring” item 점수기준
- User oriented approach : Figure 1 참고.

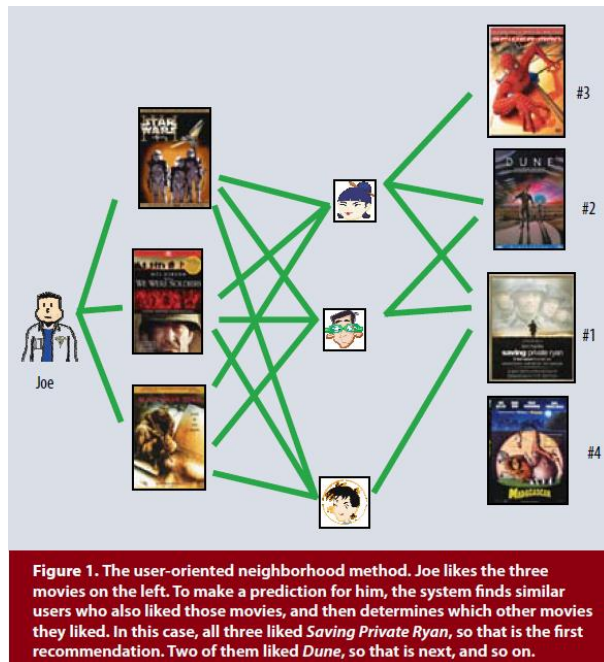


Figure1

### Latent factor models

- Rating 패턴을 factor를 추출하는 방식
- 단, 해석하기 어려운 factor가 추출될 수도 있음
- Figure 2 참고

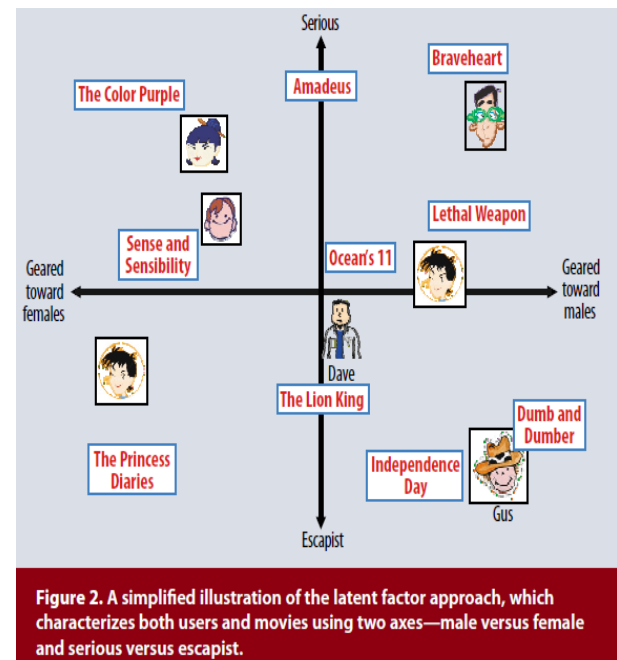


Figure2

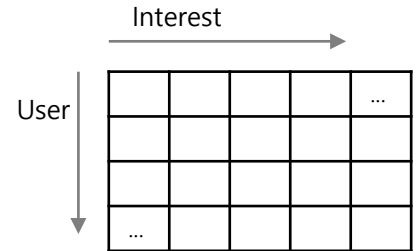
## 2. Matrix Factorization Method

■ Latent model은 Matrix Factorization을 이용

■ 보통 추천시스템은 우측 형태의 인풋 데이터 세트를 이용

➤ 하지만 이 explicit feedback 데이터는 sparsity issue가 있음

참고) rating : explicit user feedback



■ Matrix Factorization의 장점

➤ MF\*는 **implicit feedback**을 추론하므로 **sparsity issue** 극복 가능

➤ Purchase history, browsing history, search pattern, mouse movement 등 이용

### 3. Basic Matrix Factorization Model

---

- MF\*는 User와 Item을 Joint latent factor space의 dimensionality  $f$ 에 매핑하는 것
- User-Item의 상호작용을 이 공간에 inner product의 형태로 모델링하여 매핑하는 방식
  - Item  $i$ 는 특징 벡터  $q_i \in \mathbb{R}^f$ , User  $u$ 는 특징 벡터  $p_u \in \mathbb{R}^f$  와 연관있다고 가정
  - 어떤 Item  $i$ 가 특징  $q_i$ 에 있어서 positive or negative로 속성을 갖고 있는 정도
  - 어떤 User  $u$ 가 특징  $p_u$ 에 있어서 positive or negative로 속성을 갖고 있는 정도
  - 이때  $r$ 은 user  $u$ 가 item  $i$ 에 대한 평가의 근사치(approximate of rating)

$$\hat{r}_{ui} = q_i^T p_u. \quad (1)$$

- 문제는 각 item과 user를 factor vector  $q_i, p_u$ 에 대해 어떻게 매핑할 것인지를 계산하는 것

### 3. Basic Matrix Factorization Model

#### ■ SVM(Singular Value Decomposition) 이용 시 문제점

- Missing value가 많으면 SVD이용이 곤란
- Matrix에 대한 정보가 충분하지 않으면 정의하기 곤란하거나 Overfitting 위험

#### ■ 기존의 해결방식은 missing value imputation을 해서 문제를 해결

- data가 많아지면 매우 힘든(expensive한) 방법이 될 수 있음
- 부정확한 imputation은 data를 왜곡시킬 위험이 있음

#### ■ 최근의 해결방식은 **regularized modeling** 을 해서 해결

- Factor vector를 학습시키기 위해, **regularized squared error를 최소화**

$$\min_{q^*, p^*} \sum_{(u,i) \in K} (r_{ui} - q_i^T p_u)^2 + \lambda(\|q_i\|^2 + \|p_u\|^2)$$

(2) 여기서 K는 set of the (u, i) pairs  
R은 알려진 rating (training set)

- 학습한 파라미터를 regularizing하므로 관찰된 데이터에 overfitting되는 것을 극복
- 상수  $\lambda$ 는 regularization의 정도를 조절하는 상수

## 4. Learning Algorithms

### ■ Stochastic gradient descent

- $r_{ui}$ 를 예측하고 예측 에러를 계산
- Gradient의 반대 방향에서  
r에 비례하는 양 만큼 파라미터를 수정

$$e_{ui} \stackrel{\text{def}}{=} r_{ui} - q_i^T p_u.$$

- $q_i \leftarrow q_i + \gamma \cdot (e_{ui} \cdot p_u - \lambda \cdot q_i)$
- $p_u \leftarrow p_u + \gamma \cdot (e_{ui} \cdot q_i - \lambda \cdot p_u)$

### ■ Alternating least squares

- $q_i$ 와  $p_u$ 는 unknown parameter 이기 때문에, 앞의 식(2)는 convex 가 아님
- unknown parameter 중 하나를 fix하면, 이 최적화 문제는 2차식(quadratic)이 됨
- ALS 테크닉이란  $q_i$ 와  $p_u$ 를 교대로 *fixing*하는 방식을 의미
  - ✓  $p_u$ 를 fix하면, least-square 문제를 해결하는 방식으로  $q_i$ 을 재 계산함
  - ✓ 보통은 SGD가 ALS보다 빠르고 용이하나, 다음 두 경우에는 ALS가 우월
    - parallelization 을 사용할 때
    - implicit data에 집중할 때



## 5. Adding Biases

### ■ Collaborative filtering 대비 MF\*의 장점

- 다양한 데이터와 기타 application specific 요구사항을 반영해 내는 Flexibility

### ■ 식(1)은 user와 item의 상호작용을 찾아내는 것인데,

- 많은 경우 관찰된 변량은 biases 또는 intercepts와 관련이 있음
- 예를 들어, 어떤 user 들은 더 높은 점수를 주는 경향이 있고, 어떤 item은 좀더 좋은 점수를 받는 경향이 있는데, 이것이 어떤 제품이 더 좋다는 인식을 만들어 냄
- 따라서  $q_i^T p_u$  형태로 전체 rating 값을 설명하는 것은 현명하지 않음.

$$b_{ui} = \mu + b_i + b_u \quad (3)$$

Overall average rating    Observed deviation of user u and item i

$$\hat{r}_{ui} = \mu + b_i + b_u + q_i^T p_u \quad (4)$$

- Global average, item bias, user bias, user item interaction

## 5. Adding Biases

---

### ■ Squared error function을 minimizing

- Biases는 observed signal을 잡아내기 때문에 정확한 모델링이 결정적 사항

$$\min_{p^*, q^*, b^*} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2) \quad (5)$$

- 많은 작업들이 보다 정교한 bias 모델을 제안하고 있음

## 6. Additional Input Sources

### ■ Cold start problem의 해결

- User의 explicit ratings 에 관계없이 구매이력, 브라우징 기록 등 추가 정보 이용

$$\sum_{i \in N(u)} x_i$$

User u가 item에 대해 갖는 선호



$$|N(u)|^{-0.5} \sum_{i \in N(u)} x_i^{4.5}$$

Normalize 한 것

$$\sum_{a \in A(u)} y_a$$

User u의 특성에 대한 정보(성, 연령...)

- 모든 signal source 를 통합해서 아래와 같이 표현. 이때 user 표현을 더 강화

$$\hat{r}_{ui} = \mu + b_i + b_u + q_i^T \left[ p_u + |N(u)|^{-0.5} \sum_{i \in N(u)} x_i + \sum_{a \in A(u)} y_a \right] \quad (6)$$

## 7. Temporal Dynamics

### ■ 변화무쌍한 현실 세계

- 신제품이 출시되면 제품/브랜드에 대한 인식 및 인기는 지속적으로 변화
- 소비자의 선호경향도 지속적으로 진화

### ■ 모델링

- Item biases  $b_i(t)$  : Item popularity 의 변화를 시간의 함수로
- User biases  $b_u(t)$  : Natural drift in a user's rating을 시간의 함수로
- User preferences  $p_u(t)$ : user 와 item 의 상호작용하므로 user preference도 시간에 따라 변화하는 함수로 표현
- 앞의 식 4는 아래 식 7과 같이 dynamic을 포함하여 다시 표현 가능

VS

$$\hat{r}_{ui} = \mu + b_i + b_u + q_i^T p_u \quad (4)$$
$$\hat{r}_{ui}(t) = \mu + b_i(t) + b_u(t) + q_i^T p_u(t) \quad (7)$$

(특정 시간 t에서의 rating 을 예측하는 방정식)

## 8. Inputs With Varying Confidence Levels

### ■ 유의수준도 변화무쌍

- 대규모 광고 집행의 단기적 영향
- 의도적으로 rating을 높거나 낮게 주는 소비자의 문제
- 소비자 선호경향을 추론하는 방식의 추천시스템의 문제(implicit feedback)

### ■ 유의수준을 모델링에 반영

- Confidence score을 병기하는 것이 바람직함
- Confidence를  $c_{ui}$ 로 표현하면, 식(5)는 식(8)과 같이 표현될 수 있음

$$\min_{p^*, q^*, b^*} \sum_{(u,i) \in K} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

➡  
(5)

$$\min_{p^*, q^*, b^*} \sum_{(u,i) \in K} c_{ui} (r_{ui} - \mu - b_u - b_i - p_u^T q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2) \quad (8)$$

“Collaborative Filtering for Implicit Feedback Datasets”

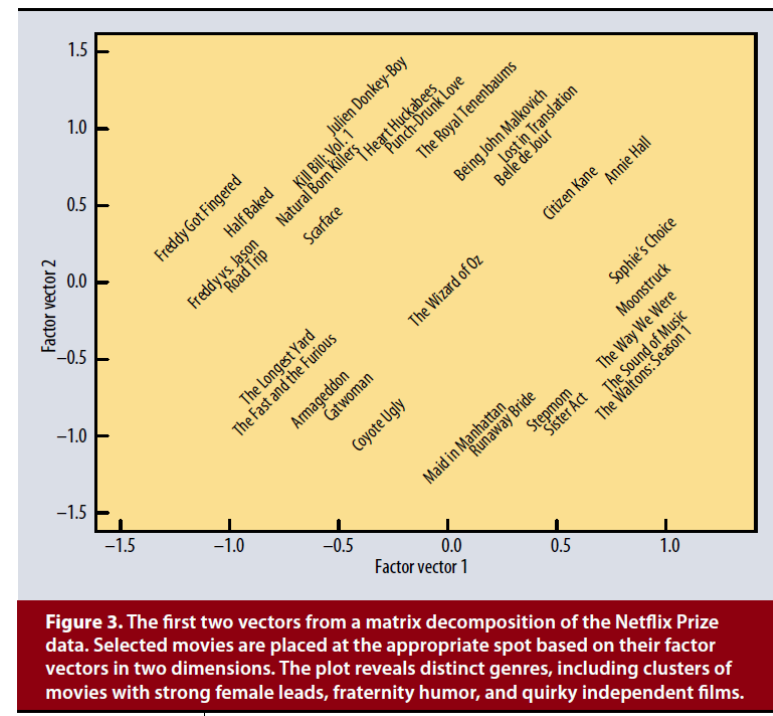
## 9. Netflix Prize Competition

### ■ 넷플릭스 공모전(2006년 실시)

- 50만명 이상 소비자가 17,000 영화에 (5점 척도) 평점을 준 1억 건의 데이터 셋
- 테스트셋 3백만 건에 대한 rating 예측치를 제출하면 넷플릭스는 RMSE 계산
- 10% 이상 성능을 개선시키면 10억, 아무도 없으면 그냥 1등에게 5만 달러
- 182개국에서 48,000팀이 데이터를 다운로드 받았음

### ■ 저자가 Matrix Factorization으로 1등한 이야기

- MF로 dimension(factor vector)을 발견함
- X축 : 저속 Comedy & Horror, 남성/청소년 타겟  
Drama & Comedy, 심각, 강한 여성
- Y축: 독립영화, 수상작 vs 정형화된 주류 영화
- 저속 & 독립영화가 가깝고,
- 심각한 여성주도 영화& 주류 영화가 가깝게 포지셔닝



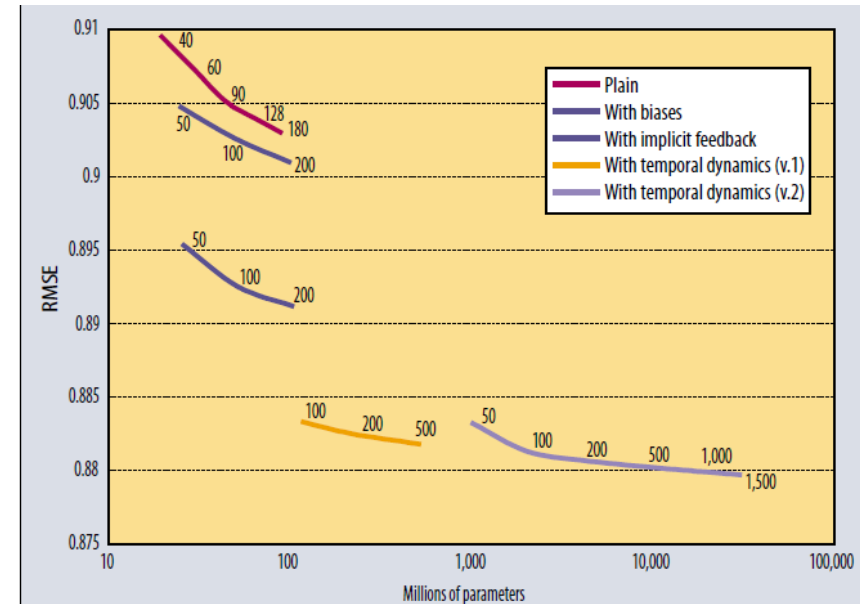
## 9. Netflix Prize Competition

### ■ 모델링 성과 분석

- 파라미터가 많아질수록 정확도 증가
- 모델이 복잡할수록 정확도 증가
- Temporal component가 특히 중요

### ■ 총정리

- Matrix Factorization은 중요한 방법론
- 기존 Nearest-Neighbor 보다 우월
- Multiple forms of feedback, temporal dynamics, 그리고 confidence level과 같은 다양한 측면 반영할수록 훨씬 더 좋아질 것임



**Figure 4. Matrix factorization models' accuracy.** The plots show the root-mean-square error of each of four individual factor models (lower is better). Accuracy improves when the factor model's dimensionality (denoted by numbers on the charts) increases. In addition, the more refined factor models, whose descriptions involve more distinct sets of parameters, are more accurate. For comparison, the Netflix system achieves  $RMSE = 0.9514$  on the same dataset, while the grand prize's required accuracy is  $RMSE = 0.8563$ .

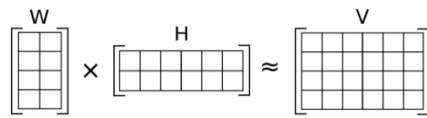




## 참고) 행렬의 인수분해

### ■ Non-negative Matrix Factorization

- 원본 행렬  $V$ 가 있다고 했을 때, 행렬 인수 분해는 이  $V$  행렬을 두개의 행렬로 분리 하는 것으로, 아래 그림은 원본 행렬  $V$  를  $V=W*H$  로 분리한 예이다



행렬  $W$ 는 다음과 같은 모양을 가지게 되고

| 책제목   | 특징1 | 특징2 | 특징3 | 특징4 |
|-------|-----|-----|-----|-----|
| 협상의법칙 | 0.9 | 0   | 0.1 | 0.2 |
| 린스타트업 | 0   | 0.8 | 0   | 0   |
| 빅데이터  | 0.2 | 0.1 | 0.8 | 0.1 |

카테고리(책제목)과 특성과의 관계

X

행렬  $H$ 는 다음과 같은 모양을 가지게 된다.

|     | 협상   | 스타트업 | 투자  | 비즈니스 | 데이터 | ... | ... |
|-----|------|------|-----|------|-----|-----|-----|
| 특징1 | 0.92 | 0    | 0.1 | 0.2  | 0   |     |     |
| 특징2 | 0    | 0.85 | 0.5 | 0.3  | 0.3 |     |     |
| 특징3 | 0    | 0    | 0.3 | 0    | 0.8 |     |     |
| 특징4 | 0    | 0    | 0   | 0    | 0   | ... |     |

원래 특성(협상, 스타트업)과 새로운 특성과의 관계

=

| 책제목   | 협상  | 스타트업 | 투자  | 비즈니스 | 데이터 | ... | ... |
|-------|-----|------|-----|------|-----|-----|-----|
| 협상의법칙 | 0.9 | 0    | 0.3 | 0.8  | 0   |     |     |
| 린스타트업 | 0   | 0.8  | 0.7 | 0.9  | 0.3 |     |     |
| 빅데이터  | 0   | 0    | 0.5 | 0    | 0.8 |     |     |

< 그림. 책 제목과, 그 책에 나온 단어의 TFIDF 값으로 이루어진 행렬  $V$  >

## 참고) SVD

### ■ Low rank assumption

- 모든 matrix는 두 matrix의 곱으로 표현이 가능함
- 이때 matrix의 rank 가 작다면 두 matrix의 rank 역시 더 작은 형태로 표현이 가능함
- n by m matrix R이 n이나 m보다 작은 k만큼의 rank를 가졌을 때, R은 n by k matrix P와 m by k matrix Q의 곱으로 표현할 수 있음. 즉  $R=PQ^T$  으로 표현됨  $r_{ui} = p_u \cdot q_i$
- User u가 item i의 점수를 주는 방식은 user u 의 item들에 대한 숨겨진 interest  $p_u$  와 그에 대응하는 item들의 숨겨진 특성  $q_i$  에 의해 결정된다는 사실을 알 수 있음

